

Присваивание копированием и самоприсваивание

1. В чём проблема?

При сдаче ЛР1 многие из вас получили замечание приблизительно следующего содержания «Оператор присваивания копированием должен проверять, а не равен и текущий объект присваиваемому».

Что это значит? Приблизительно то, что вы видите в строке 30 примера 1: объекту `first` присваивается он же. В чём смысл этого кода? Его нет ☺ В таком варианте он и не встречается (разве что в случае ошибки). А вот в более завуалированном, когда программы становятся сложнее, в них появляется много указателей и ссылок – вполне.

Пример 1 служит исключительно для демонстрации самоприсваивания. С точки зрения качества кода он очень плох, т.к. определение и описание класса слеплены воедино, а также у класса отсутствуют конструктор копирования и деструктор.

Пример 1. Самоприсваивание

```
1  class MyClass
2  {
3      float* values;
4      size_t size;
5  public:
6      MyClass() : values(nullptr), size(0) {}
7
8      MyClass(size_t size_, float* const& values_) : size(size_)
9      {
10         values = new float[size];
11         for (size_t i = 0; i < size; ++i)
12             values[i] = values_[i];
13     }
14
15     MyClass& operator=(const MyClass& other)
16     {
17         size = other.size;
18         delete[] values;
19         values = new float[size];
20         for (size_t i = 0; i < size; ++i)
21             values[i] = other.values[i];
22         return *this;
23     }
24 };
25
26 int main()
27 {
28     float* values = new float[4]{ .3, 1.5, 3., 7.2 };
29     MyClass first(4, values);
30     first = first;
31     return 0;
32 }
```

Но в чём проблема? Подумаешь, ну присвоим мы `first` его же значение. Конкретно в примере 1 описан случай, когда самоприсваивание для данных фатально. В обычном случае при присваивании нас не интересует, что находится в текущем объекте: мы освобождаем ресурсы, если это нужно, и перезаписываем на их место значения из объекта, переданного в качестве параметра. В текущем примере `other` совпадает с текущим объектом, поэтому

строкой 18 освобождается память, на которую ссылается и `other`. В итоге значения, которые были, теряются, на их месте остаётся мусор.

2. Что делать?

Первый (и самый нехороший) вариант – убрать `&` около параметра, т.е. передавать параметр не по ссылке, а по значению. Для корректной работы должен быть корректно определён конструктор копирования (т.е. с глубоким копированием). Тогда при использовании оператора присваивания для аргумента будет вызван конструктор копирования, и `delete` затронет только память текущего объекта, а в `other` будет его глубокая копия.

Этот вариант плох по двум причинам:

- 1) будет проделана лишняя работа,
- 2) если объект большой, придётся потратить много лишней памяти.

Второй (хороший) вариант – проверить, не указывает ли `this` на ту же область памяти, что и `other`. Такой вариант приведён в примере 2. Поскольку `this` – это указатель, сравнивается он не с `other`, а с адресом `other`, который мы получаем с помощью оператора взятия адреса `&`. И этот вариант спасёт от ситуации из примера 1, когда объекту присваивается он сам.

Пример 2. Присваивание копированием с проверкой адресов

```
1 MyClass& operator=(const MyClass &other)
2 {
3     if (this != &other)
4     {
5         size = other.size;
6         delete[] values;
7         values = new float[size];
8         for (size_t i = 0; i < size; ++i)
9             values[i] = other.values[i];
10    }
11    return *this;
12 }
```

Третий вариант тоже плохой, хотя на первый взгляд он похож на второй, а оттого кажется хорошим. Он приведён в примере 3 и «защищает» от копирования не только того же самого объекта, но и другого, равного текущему.

Пример 3. Присваивание копированием с проверкой объектов

```
1 MyClass& operator=(const MyClass &other)
2 {
3     if (*this != other)
4     {
5         size = other.size;
6         delete[] values;
7         values = new float[size];
8         for (size_t i = 0; i < size; ++i)
9             values[i] = other.values[i];
10    }
11    return *this;
12 }
```

Для того, чтобы сравнивались объекты, производится разыменование указателя. Для того, чтобы код сработал, требуется определить оператор(ы) сравнения для объектов класса. Для данного класса они приведены в примере 4. Стоит отметить, что перегрузка этих операторов требуется довольно часто, а конкретно этом случае можно было ограничиться одним оператором `==`, так что этот вариант плох не тем, что для него требуется написать дополнительные функции.

Пример 4. Присваивание копированием с проверкой объектов

```
1  bool operator==(const MyClass &other)
2  {
3      if (size == other.size)
4      {
5          for (size_t i = 0; i < size; ++i)
6              if (values[i] != other.values[i])
7                  return false;
8          return true;
9      }
10     return false;
11 }
12
13 bool operator!=(const MyClass &other)
14 {
15     return !(*this == other);
16 }
```

Плох вариант именно сравнением объектов.

Как понять, что два объекта равны друг другу? Пройти по всем их полям, сравнить все их компоненты и убедиться, что все они равны. В нашем случае – сравнить `size` и все элементы в `values`.

В случае равенства объектов мы бы обошли все значения. В случае перезаписи значений мы тоже обошли бы все значения. Получилось «шило на мыло» (с поправкой на ситуации, когда операции перезаписи очень «дорогие»).

А в случае, когда объекты равны не полностью, производится гораздо больше работы, чем во втором варианте. Например, если бы был объект, который отличался от текущего только последним значением в `values`, то пришлось бы пройти по всему содержимому при сравнении, а потом ещё раз при перезаписи.

Вот так вот вариант, который кажется хорошим в теории, оказывается не очень хорошим на практике ☺

Вывод: в операторе присваивания копированием не забывайте сравнивать адреса текущего объекта и присваиваемого.